Exercise 6: Spec-Driven Development with Spec Kit

Session: 6 — Real Development Workflows
Duration: 45 minutes
Project: `web`, a new frontend for the BookStore API you already know.

Goal

You run a spec-driven flow from a one-sentence idea to a reviewed task list. Spec Kit writes the files: a spec, a plan and a task list. You read them, answer its questions, and check what they claim against the running API. You will **not** write any application code. Exercise 7 builds what you specify here, with a team of agents working in parallel.

Spec Kit is a practice tool here: it shows every step of a good spec. It is not a recommendation. Your team's version can be much lighter.

Why a frontend

The BookStore API has no user interface. So nothing tells you what an empty list looks like, what a failed request shows, or which fields a book page has. Those are decisions. Decisions are what a specification is for.

A frontend also splits into parts that different agents can build at the same time. Exercise 7 depends on that.

Start here

Terminal 1, in the `bookstore` folder, runs the backend:

```
go run .
```

Terminal 2, in the `web` folder inside `bookstore`, runs Claude. The `..` gives Claude your backend's code:

```
claude --add-dir ..
```

Terminal 3, a plain shell in `web`, for `curl` and `git`. Check the backend and Spec Kit there:

```
curl http://localhost:8080/api/books
specify check
```

The first prints JSON, the second reports no problems. Do not start the frontend: there is none yet.

Training mode is off in `web`: Claude answers directly. Stuck? Type *just tell me*.

Tasks

1. Create the spec and find what it leaves open

You create `specs/001-*/spec.md`. The spec says *what* the frontend does and *why*. Not how.

1. **Start the spec**. In terminal 2, run `/speckit-specify` with this text, exactly as written:

   > A web UI for the BookStore API. Users can browse books and open a book to see its details and its author. The API is already running behind `/api`.

   The command takes one to two minutes. Do not wait for it. Claude does not know `/speckit-specify`? Run the Spec Kit steps in `preparation.pdf` again, then restart Claude.

2. **Read the constitution while it runs**. Open `.specify/memory/constitution.md`. The constitution holds the rules that every spec, plan and task must follow. It is written for you. It has six principles on one page. Answer two questions in one written sentence. What does the constitution say about the API contract? Where does it say the contract comes from?

3. **Find the gaps**. When the command is done, open `specs/001-*/spec.md`. Scroll to **Assumptions** at the end. This is where Spec Kit writes what it invented. Nothing has called the API yet, so each assumption is a guess. Now read the **Requirements** section with three questions in mind:

   - How does a visitor reach page 2 of a long list?
   - What does the list show when a page has no books?

- What does the page show for a book id that does not exist?

Write down which of the three the spec answers. Expect one or none.

Done when: `spec.md` exists, you wrote the constitution answer, and you wrote which of the three questions the spec answers.

2. Save the spec, then answer the clarify questions

You change `specs/001-*/spec.md` in place. No new file.

1. **Save the spec first**. In terminal 3:

```
git add specs
```

   You do not commit. Later, `git diff` shows the difference between this saved version and the changed file. Task 3 reads that diff.

2. **Run** `/speckit-clarify`. It asks up to five questions about what the spec leaves open, one at a time, and waits for each answer. Answer fast. This is a first draft, not a perfect spec.

   One rule: **if you are guessing, write the word "guess" in your answer.** Your words go into the spec exactly as you type them. So the label stays with the guess. Example answer:

   > 20 books per page (guess, not checked against the API)

   Zero questions? Then Spec Kit found nothing open. Go on to task 3.

3. **Stop**. Do not read the spec yet. You read it in task 3, while a slow command runs.

The command does more than list your answers. Every answer is also written into the section where it belongs: Functional Requirements, User Stories, Data Model, Success Criteria or Edge Cases. Where an answer contradicts an old sentence, the old sentence is replaced.

Done when: `/speckit-clarify` is finished, and every answer you guessed contains the word "guess".

3. Create the plan, and review the spec while it runs

You create five files under `specs/001-*/`: `plan.md`, `research.md`, `data-model.md`, `contracts/` and `quickstart.md`. Together they are the plan. The plan says *how* the frontend gets built.

1. **Start the plan**. In terminal 2, run `/speckit-plan`. It runs two to four minutes without you. Do not wait. Do not edit the spec while it runs: the command is reading that file.
2. **Read the diff**. In terminal 3:

```
git diff -- specs
```

   The `## Clarifications` section near the top lists the questions you were asked. The rest of the diff shows where your answers went. Look for three things:

   - **Your guesses.** Find each answer with the word "guess". It now looks like a fact to everyone who reads the spec later, including the agents in Exercise 7.
   - **Removed lines.** Lines that start with `-` outside the Clarifications section. An answer contradicted an old sentence, so Spec Kit removed it. You only see this in the diff.
   - **Answers in a place you did not expect.** One answer can change a user story, add an edge case and change the data model.

3. **Write down one thing you disagree with**. Do not fix it yet. In a real project it goes back into the spec before the plan is used.

Done when: the five files exist, and you wrote down one line of the spec you disagree with.

4. Predict, then check the plan against the API

You produce a written list of differences between `research.md` plus `contracts/` and the real API.

`/speckit-plan` did its own research. It wrote what it thinks the API does into `research.md` and `contracts/`. Exercise 7 builds against those files. So check them. The paths you call here are the paths the frontend will use: it calls `/api/...`, and Vite forwards that to port 8080 (`vite.config.js`).

1. **Predict first**. Before you run anything, write down two predictions:
   - `GET /api/books/1` returns: a book object, or something else?

- The first page of the list is page number: 0 or 1?

A wrong prediction is fine. A missing prediction is the only failure.

2. **Call the API**. In terminal 3:

```
curl -s "http://localhost:8080/api/books?page=0&size=3"
curl -s "http://localhost:8080/api/books?page=1&size=3"
curl -s "http://localhost:8080/api/books/1"
```

   On Windows PowerShell, type `curl.exe` instead of `curl`. Compare the output with your two predictions.

3. **Check the two files**. Open `specs/001-*/research.md` and the file in `specs/001-*/contracts/`. Compare them with the curl output:

   - **Shape**: what do they say `GET /api/books/{id}` returns? Is that what you saw?
   - **Paging**: which number is the first page, and what happens with page 0? Is that what you saw?
   - **Inventions**: endpoints, fields or status codes that are not in your curl output. A wrong line in a contract does the most damage, because code gets built on it.

   Write down every difference. An empty list is a valid result. In task 6, `/verify-exercise 6` compares this list with your backend.

Done when: both predictions were written before the first curl, and you have a list of differences (possibly empty).

5. Create the task list and check the `[P]` marks

You create `specs/001-*/tasks.md`. The task list holds small steps, in order, for building the frontend.

1. **Start the task list**. In terminal 2, run `/speckit-tasks`. It takes one to two minutes. While it runs, look at your task 4 list: which difference would do the most damage once it is code?
2. **Read the format**. Open `specs/001-*/tasks.md`. Each task looks like `[ID] [P?] [Story]`. `[P]` means: different files, no dependencies, so it can run at the same time as other `[P]` tasks. `[US1]` and `[US2]` name the user story. Find the sections **Parallel Opportunities** and **Parallel Team Strategy** near the end. That is the work split for the agents in Exercise 7.

3. **Check for a collision**. Pick two `[P]` tasks from **different** user stories. Do they write the same file? Then check the shared files: the API client, the stylesheet, `index.html`. The constitution (principle IV) says these belong to the foundation phase, not to a story. If a story task writes one of them, two agents will edit the same file at the same time. One of them loses its work without an error (*Can These Run at the Same Time?*). Write down: *collision: yes or no*, and the file name if yes.

Done when: you wrote down *collision: yes* with a file name, or *collision: no*.

6. Commit and run the check

1. **Commit**. In terminal 3:

```
git add specs
git commit -m "spec: bookstore-web frontend"
```

   Exercise 7 starts from this commit.

2. **Start the check**. In terminal 2, run `/verify-exercise 6`. This is a course command. It compares what the plan says about the API with what your backend really does. It asks you for two file names: `research.md` and the contract file. Give them. It calls your backend, compares, and reports. Read the report during the closing round.

Done when: `git log` shows the commit, and `/verify-exercise 6` is running.

Bonus (only if time remains)

Run `/speckit-analyze`. It reads `spec.md`, `plan.md` and `tasks.md` and reports where they contradict each other. It writes no files. Pick one finding you agree with and one you do not. Then compare the report with your task 4 list. Is a difference you *know* is real missing from the report? Then you have seen the limit of this command. The three files can agree with each other and still be wrong about the API.

Closing round

The trainer asks the room. Have these answers ready:

- Which `/speckit-clarify` question did you not expect?
- Which of your answers changed a part of the spec you did not expect?

- Did your two predictions in task 4 hold? What did the plan get wrong about the API?
- Did you find a `[P]` collision? Which file?
- Everyone started from the same sentence. Where does your spec differ from the spec of the person next to you? Would you have seen that difference without a spec?
- A year from now, a colleague changes the book list. Which file in `web/specs/001-*/` should they update, and which files are history?

What you should have

```
web/specs/001-*/
  spec.md            intent, with your clarify answers written in
  plan.md            approach
  research.md        what the plan thinks the API does
  data-model.md      entities
  contracts/         the API contract, as the plan wrote it
  quickstart.md      how to verify
  tasks.md           the work list, with [P] marks
```

No application code. `/speckit-implement` is Exercise 7.